Nexiora

Smart Traffic Management System

A Beginner's Guide to Every Feature:
What It Does, How It Works, What You See, and How It Was Built

This guide explains the whole project in plain, simple language - no coding background needed. Every section follows the same pattern: what the feature does, how it works step by step, what you actually see on screen, and how it was built.

Honesty note: this is a college final-year project built without physical hardware. Cameras are recorded traffic videos, traffic lights are drawn on screen, RFID is a virtual tag check, GPS is a simulated ambulance, and the other city junctions are simulated. The vehicle detection, the AI model, and the counting are real and run on real video.

Contents

1. The Big Picture - How Everything Fits Together
2. Watching the Road - Vehicle Detection & Counting
3. Fair Counting - PCU-Weighted Traffic Load
4. Smart Traffic Signals - Adaptive Signal Control
5. Proving It Works - Fixed-Time Comparison & Savings
6. Catching Rule-Breakers - Red-Light Violation Detection
7. Ambulance Priority - RFID Check, Green Corridor & Nearest-Unit Alert
8. Seeing Into the Future - Traffic Prediction
9. Spotting Trouble - Accident Detection & Rerouting
10. Clean Air Matters - Air Quality & Eco Mode
11. Keeping Score - Reports & History
12. The Public Website
13. The Controller's Control Room (Dashboard)
14. Behind the Scenes - Login, Database & How It All Runs

1. The Big Picture

Think of Nexiora as a brain for a traffic junction. A camera (a recorded video, in this project) watches each of the 4 roads coming into the junction. The brain (a Python program) looks at the video, counts vehicles, decides which road should get a green light and for how long, watches for accidents and rule-breakers, and writes everything down in a notebook (a database). A website then reads that notebook and shows the results to two kinds of people:

- **The public** - anyone can open the website and see live traffic, the map, predictions, and air quality, with no login needed.
- **The controller** - a traffic authority staff member who logs in to see everything in detail, watch live video, approve ambulance requests, and override signals by hand if needed.

> **What you'll actually see**
>
> • Open the website and you immediately see 4 junction cards (J1-J4), each showing its traffic-light colours, how busy it is (Low/Medium/High), and the local air quality.
>
> • Behind the scenes, this same information is refreshing automatically every 2 seconds - no page refresh needed.

How we built it, in simple steps:

1. Wrote a Python program that opens 4 traffic videos (one per road/lane) and reads them frame by frame, like flipping through a photo album very fast.
2. Fed each frame into an AI model that finds and counts vehicles (explained in Section 2).
3. Wrote a 'decision maker' that looks at the counts and decides signal timing (Section 4), checks for violations (Section 6) and accidents (Section 9).
4. Saved all the results into a simple local database file on the computer - no internet or cloud account needed.
5. Built a website (using React) that asks the database 'what's new?' every 2 seconds and redraws the screen with the latest numbers.

2. Watching the Road - Vehicle Detection & Counting

Project objectives: intelligent traffic monitoring, real-time vehicle counting per lane

What it does

Looks at each lane's video and figures out exactly how many vehicles are there right now, what type each one is (car, bike, auto-rickshaw, bus, or truck), and tracks each vehicle so it isn't counted twice as it moves across the screen.

How it works, step by step

1. A video frame (one photo from the video) is given to an AI model called **YOLOv8** - think of it as a very fast, trained 'spot the vehicle' game player. It draws a box around every vehicle it finds and labels what type it thinks it is.
2. This AI model was **trained specially for Indian traffic** using a real dataset called UVH-26 (thousands of labelled Indian road photos), so it recognises auto-rickshaws and two-wheelers, not just cars like a generic model would.
3. A second tool called **ByteTrack** gives each detected vehicle an ID number and follows it from frame to frame - like putting a sticky note with a number on every car the moment it appears, so the system knows 'this is the same car as last frame', not a new one.
4. A drawn boundary line (called an ROI - 'region of interest') marks the real lane area on screen. Only vehicles whose bottom-centre point is inside that boundary are counted for that lane - this stops vehicles in a neighbouring lane from being counted by mistake.

What you'll actually see

- On the live video feed, every vehicle has a green box drawn around it with its type and ID number, e.g. 'car #482'.
- Each lane shows a running total like '7 vehicles' directly on the video overlay and on the dashboard.
- If the camera feed is temporarily unavailable, the system says so clearly instead of freezing or crashing.

How we built it

1. Downloaded and converted the UVH-26 Indian traffic dataset into a format the AI model understands.
2. Trained a YOLOv8 model on a home GPU (laptop graphics card) to recognise 5 vehicle types: car, bike, auto, bus, truck.
3. Measured how accurate it is (a score called 'mAP' - about 90% accuracy overall) and compared it against an untrained, generic model to prove the training actually helped.
4. Connected the trained model to live video so it runs continuously, plus the ByteTrack tracker so each vehicle keeps one consistent ID as it moves.

3. Fair Counting - PCU-Weighted Traffic Load

What it does

Simply counting '5 vehicles' isn't fair - 5 bikes take up far less road space than 5 trucks. So the system also calculates a weighted number called **PCU** (Passenger Car Unit), a standard traffic-engineering idea: a car counts as 1 unit, a bike as 0.5 units (half a car's worth of road space), and a bus or truck as 3 units each.

How it works

1. Every detected vehicle already has a type label from Section 2 (car, bike, auto, bus, truck).
2. The system multiplies the count of each type by its PCU weight and adds them all up.
3. Example: 2 cars + 4 bikes + 1 truck = (2x1.0) + (4x0.5) + (1x3.0) = 7.0 PCU, even though the raw vehicle count is only 7 vehicles - here they happen to match, but a lane full of trucks will always show a much higher PCU than the same number of bikes.

What you'll actually see

- Every lane shows both numbers side by side, e.g. '6 vehicles, PCU 9.0' - so you can see both the raw count and the real road-space load.
- The junction cards show a congestion badge (Low / Medium / High) calculated from this PCU total, not just the raw count - a junction full of trucks is correctly shown as more congested than one full of bikes, even with the same vehicle count.

How we built it

1. Set the 5 PCU weight values in one central settings file, based on standard Indian Road Congress guidelines, so they can be tuned later without touching any other code.
2. Made every part of the system - the signal controller, the website, the predictions - read traffic 'load' from this same PCU number, so the whole project is consistent.

4. Smart Traffic Signals - Adaptive Signal Control

Project objectives: adaptive signal control - longer green for busy lanes, less waiting for quiet lanes

What it does

Instead of a traffic light that always gives each road exactly 30 seconds no matter what, this system looks at how busy each lane actually is right now and gives busier lanes a longer green light, and quiet lanes a shorter one - automatically, every cycle.

How it works, step by step

1. Every lane's current PCU load (Section 3) is fed into a formula: *green time = a base amount of seconds + extra seconds for every unit of PCU waiting*, with a minimum and maximum so no lane gets an unfairly short or endlessly long green.
2. After the green light's time runs out, a short yellow light warns drivers before the light turns red - this never changes, for safety.
3. The system then picks which lane goes green next: normally the lane with the most PCU waiting, but if any lane has been waiting too long (even a quiet one), it jumps the queue so no lane is ever ignored forever.
4. One safety rule is checked on every single update, with no exceptions: only one lane can ever be green or yellow at the same time. The other three are always red.
5. The system can also switch modes: normal ('adaptive'), rush hour ('peak', using the prediction feature), pollution-friendly ('eco'), emergency vehicle passing through, accident response, or a manual override by the human controller.

What you'll actually see

- On the video overlay and the dashboard, you see a traffic-light icon per lane with the real colour (red/yellow/green) and a countdown timer, e.g. 'Green: 17s left'.
- A plain-English log line is written for every decision the system makes, e.g. '[J1] lane3 -> GREEN for 22.0s (load=14.5, mode=adaptive)' - useful for explaining exactly why the system did what it did.

How we built it

1. Wrote a 'SignalController' program for each junction that holds the current state (which lane is green, for how long) and updates it once per second.
2. Wrote over 70 automated tests that check the safety rules can never be broken - for example, a test that tries to force two lanes green at once and checks the system refuses.
3. Connected the live PCU counts from Section 3 directly into this controller, so signal timing reacts to real traffic within seconds.

5. Proving It Works - Fixed-Time Comparison & Savings

Project objectives: measurably reduce waiting time, idle fuel and emissions

What it does

It's not enough to say 'our smart signals are better' - you have to prove it with numbers. So a second, imaginary traffic light runs silently alongside the real one, using the old-fashioned fixed-time method (always the same number of seconds per lane) on the exact same traffic. The system then compares the two.

How it works

1. Both the adaptive (smart) and fixed-time (old-fashioned) signal timers watch the same live vehicle counts, but make their own independent decisions - the fixed-time one never actually controls the real lights, it only runs the maths in the background for comparison.
2. Every second, every vehicle sitting at a red or yellow light adds one 'waiting vehicle-second' to that method's running total.
3. Average waiting time = total waiting vehicle-seconds divided by how many vehicles were served - a simple, honest average.
4. Wasted fuel and CO2 are estimated from the waiting time, using a standard idling-fuel-burn rate (a labelled assumption, not a measured fact, since there's no real fuel sensor).

What you'll actually see

- The dashboard's Performance page shows a side-by-side chart: 'Adaptive: 12.4s average wait' vs 'Fixed-time: 19.8s average wait', plus estimated litres of fuel and kilograms of CO2 saved per day.
- These are real, measured numbers from the actual running system, not invented - the project is careful to only claim savings it can show the maths for.

How we built it

1. Built a simple fixed-time controller that always gives every lane the same cycle, for a fair 'before' comparison.
2. Built a shared waiting-time tracker used by both controllers, so the comparison is apples-to-apples.
3. Set the fuel/CO2 conversion numbers in the settings file and labelled them clearly as assumptions in the project's documentation, as required by academic honesty.

6. Catching Rule-Breakers - Red-Light Violation Detection

Project objectives: AI red-light violation detection with recorded evidence

What it does

Automatically catches any vehicle - car, bike, auto, bus or truck - that crosses the stop line while its lane's light is red, takes a photo as evidence, and logs it with a unique vehicle tracking number.

How it works, step by step

1. A stop-line position is marked for each lane (one pixel row on the video).
2. For every tracked vehicle, the system remembers where its bottom edge was in the previous frame and where it is now.
3. If that bottom edge was above the stop line last frame and is now past it **while the light is red**, that's a violation - caught at the exact moment of crossing, not before or after.
4. An authorised emergency vehicle that's currently been given priority is never flagged, even if it must cross on red to get through.
5. The same vehicle is only ever flagged once, even if it lingers near the line.

What you'll actually see

- On the video, the violating vehicle gets a red box and the word 'VIOLATION' written above it for a few seconds.
- A full snapshot of the moment, plus a close-up crop of just the vehicle, is saved as image files - real evidence, not just a log line.
- A new row is added to the database and shown on the controller dashboard's Violations page: lane, vehicle type (car/bike/auto/bus/truck), a random/unique tracking number, and timestamp - filterable and exportable as a table with charts by lane and time of day.

How we built it, and a real fix we made

1. Wrote a 'RedLightDetector' that watches every tracked vehicle's position each frame and compares it to the stop line and the current light colour.
2. Added an evidence-saving step using the computer's image library to crop and save the photo the instant a violation is confirmed.
3. Added a 'demo mode' that holds one lane red for 30 seconds on purpose, so a violation can be shown on stage during a presentation without waiting for a real red light.

7. Ambulance Priority - RFID Check, Green Corridor & Nearest-Unit Alert

Project objectives: automatic priority for authorised emergency vehicles, live ambulance tracking with a green corridor, faster emergency response

What it does

Lets a real, registered ambulance get a clear run of green lights across the city - but **only** after proving it's a genuine, authorised vehicle, not just because something that looks like an ambulance appears on camera. This is split into three connected parts.

Part A - Checking the ambulance is real (virtual RFID)

1. Since there's no physical RFID scanner in this project, a driver 'logs in' as a specific ambulance and enters a tag ID number - this stands in for scanning a real RFID chip on the vehicle.
2. An emergency request can also come from the camera itself spotting something that looks like an ambulance, from the controller's dashboard, or from a simulator script - but every single one of these is only a **request**, nothing more.
3. The system checks the vehicle ID and tag ID against a list of registered emergency vehicles. Only an exact match gets approved; everything else is rejected with a reason shown to the driver, and a security alert is raised.
4. This matters: a camera alone can never grant priority across multiple junctions - only a verified tag can. This stops anyone from just pretending to be an ambulance to skip traffic.

Part B - Clearing the road ahead (green corridor)

1. Once approved, the system calculates the fastest route to the destination, avoiding any roads currently blocked by an accident.
2. For each junction coming up on that route, it estimates: how many seconds until the ambulance arrives, and how many seconds that junction's own queue needs to clear out of the way.
3. If the ambulance will arrive before the queue would naturally clear, that junction switches early to give the ambulance's direction a green light - clearing the jam before it gets there, not panicking at the last second.
4. For safety, the light always passes through yellow first, never straight from red to green, and at most the next 2 junctions ahead are held at once.
5. Once the ambulance has passed, that junction goes back to normal and serves whichever lane has been waiting longest.

Part C - Telling the nearest ambulance about an accident

1. When an accident is confirmed (Section 9), the system looks at every registered emergency vehicle's home base and calculates the straight-line distance from each one to the accident.
2. The closest available unit gets a private notification on their driver screen: 'Accident near [junction name] (1.2 km from your base) - you're the nearest available unit.'

3 This closes the loop - the system doesn't just detect an accident, it tells the right people to go deal with it.

What you'll actually see

- The driver's screen shows: pending -> approved (with route, live map, and the list of junctions being held green for them) -> or rejected (with a clear reason, e.g. unknown vehicle ID).
- The public map shows the ambulance's live position moving along its route, with an on-screen banner at any junction currently being held for it.
- After the trip, a record is saved showing how many seconds were saved compared to no green corridor at all - this can be averaged over many simulated trips into a chart for the project report.

How we built it

1 Seeded a small list of registered ambulances/fire vehicles with tag IDs and a home junction.

2 Wrote a route-finder using the city's road map (modelled as a graph of junctions and roads) that always finds the fastest currently-open path.

3 Wrote a 'GreenCorridor' program that ticks once a second, recalculating ETA vs queue clearance time for every junction on the route and deciding when to pre-empt.

4 Fixed a real bug found while testing: the green corridor could only ever reach the main junction's traffic light, not the simulated ones elsewhere in the city, because they lived in a separate part of the program - connected them properly so priority now works across the whole simulated city, verified with a real end-to-end test run.

8. Seeing Into the Future - Traffic Prediction

Project objectives: ML prediction of future congestion - peak hours and next 5-10 minutes

What it does

Uses machine learning (a computer program that learns patterns from past data, rather than being told exact rules) to guess two different things: which hours of the day are usually the busiest, and what traffic will look like in the next 5 to 10 minutes.

How it works - Part A: Peak hours

1. The system keeps a history of traffic counts logged every minute, plus 30 days of realistic sample data generated to fill in gaps (clearly labelled as simulated, not pretending to be real).
2. Two different learning methods are tried - a simple one (Linear Regression) and a more flexible one (Random Forest) - and their accuracy is measured and compared honestly.
3. The better method predicts traffic for all 24 hours of the next day; the busiest 20% of those hours are marked as 'peak hours'.
4. When a peak hour arrives, the signal controller automatically switches to 'peak' mode (Section 4) to get ahead of the rush instead of reacting to it.

How it works - Part B: Next 5-10 minutes

1. The system looks at each lane's last several minutes of traffic (how it's trending: rising, falling, steady) plus the time of day and day of week.
2. It predicts whether that lane will be Low, Medium, or High in 5 minutes and in 10 minutes, refreshing this prediction every 60 seconds.
3. If a lane is predicted to go 'High' soon, the signal controller adds a few bonus seconds to that lane's next green light *before* it actually gets busy - preventing the jam rather than just reacting to it.

What you'll actually see

- The Predictions page shows a 24-hour chart with the predicted busy hours highlighted.
- Each lane shows a simple tag like 'In 5 min: Medium, In 10 min: High'.
- An alert pops up (at most once every 10 minutes per lane, so it doesn't spam) warning that a lane is about to get busy.

How we built it

1. Generated realistic synthetic traffic history (with believable rush-hour patterns) since there wasn't months of real historical data available.
2. Trained and compared Linear Regression and Random Forest models for both the hourly and minute-level predictions, measuring their error honestly (MAE, RMSE, R-squared - standard accuracy scores) instead of just picking one and assuming it works.
3. Connected the winning model's live predictions back into the signal controller, so prediction isn't just a chart - it actually changes real signal decisions.

9. Spotting Trouble - Accident Detection & Rerouting

Project objectives: accident detection, congestion-impact prediction, alternative-route suggestions

What it does

Watches the video for signs that look like an accident, warns the controller, predicts how much extra traffic delay it will cause, and automatically suggests detours around the blocked road.

How it works, step by step

1. **Rule 1 - stuck vehicle:** if a vehicle stays stationary while its own light is green for an unusually long time, that's suspicious - a vehicle that's fine would have driven off on green.
2. **Honest limitation fixed:** the recorded video has its own real-world traffic lights that this project's virtual signal doesn't know about, so a car legitimately stopped at a real red light used to look identical to a stuck car. The fix: a suspected vehicle now has to survive a *full green-red-green cycle* of the virtual signal still not moving before it's confirmed as a real accident - a real red-light stop clears well before that, but a genuinely stuck vehicle doesn't.
3. **Rule 2 - sudden collision:** if two tracked vehicles suddenly overlap heavily and both sharply drop speed within about a second, that pattern also raises suspicion.
4. A suspicion only becomes a confirmed accident after it holds up for a set number of seconds, to avoid false alarms from a single bad camera frame.
5. Once confirmed: the road is marked blocked, that approach switches to 'accident' mode, alternative routes are calculated for traffic that would have used that road, an alert is raised with the suggested detour, and any active ambulance route is automatically re-planned around the blockage.
6. The controller can also manually report or resolve an accident with one click, and a simulator can trigger a practice accident for demo purposes.

What you'll actually see

- The video shows an orange 'ACCIDENT?' box the moment something looks suspicious, and a full red accident banner once it's confirmed.
- A snapshot image is saved as evidence.
- The dashboard's Accidents page shows active and resolved accidents, the predicted extra delay in minutes, the suggested detour on a mini map, and a 'Mark resolved' button.
- The nearest ambulance also gets notified automatically (Section 7, Part C).

How we built it

1. Wrote simple, explainable rules rather than training a separate AI model, since no accident-image dataset was available - and said so honestly in the project documentation rather than pretending otherwise.
2. Added the 'must survive a full signal cycle' rule after watching the system raise false accident alarms during real testing, then wrote an automated test that specifically proves a car stuck only during one red-light stop is never falsely confirmed.

3 Connected confirmed accidents to the road-network router, so detours are calculated the same way as any other route in the system, not as a separate one-off feature.

10. Clean Air Matters - Air Quality & Eco Mode

Project objectives: air-quality monitoring and eco-friendly control

What it does

Checks the air quality near each junction and, when pollution is high, switches the signal controller into a gentler 'eco' mode that reduces how long vehicles sit idling with their engines running.

How it works

1. Every few minutes, the system fetches real air quality data (PM2.5, PM10, NO2, CO) from a free weather-and-pollution service for each junction's location; if that service is unavailable, it clearly falls back to a labelled simulated reading instead of pretending to have real data.
2. The reading is converted into a simple AQI (Air Quality Index) score and a plain label like 'Fair' or 'Moderate'.
3. If the AQI reaches a poor level, the junction automatically switches to 'eco' mode: shorter overall cycles and a lower maximum-wait limit, so fewer vehicles sit idling and polluting for a long time.
4. An 'air quality' alert is raised, but at most once per hour per junction, so it doesn't spam the alert feed.

What you'll actually see

- Every junction card shows an AQI badge, e.g. 'AQI 2 (Fair)'.
- The Air Quality page shows pollutant levels and a 24-hour trend chart per junction.
- When eco mode kicks in, the junction's mode badge changes to 'eco' and the dashboard shows this clearly.

How we built it

1. Signed up for a free tier of a weather/pollution API and stored the key securely outside the code (never hard-coded or shared).
2. Wrote a fallback simulator so the feature still demonstrates correctly even without an internet connection or API key.
3. Added an 'EcoModeManager' that checks AQI on a timer and flips the right junction's signal controller into eco mode automatically.

11. Keeping Score - Reports & History

Project objectives: historical records and reports on traffic patterns and system performance

What it does

Turns all the raw data the system has collected into readable daily summaries, charts, and downloadable reports - useful for a project viva, or for a real traffic authority reviewing performance over time.

How it works

1. A report-building script gathers everything recorded for a chosen day (or date range): traffic counts, wait times, violations, accidents, emergency trips, and air quality.
2. It produces a professional PDF with charts and captions, plus raw CSV spreadsheet files for anyone who wants to analyse the numbers themselves.
3. A 'fill in demo data' option can generate a realistic past week of records in one go, useful for showing the reports feature without needing to run the live system for a whole week first.

What you'll actually see

- The controller dashboard's Reports page lists daily summaries with a trend chart, and buttons to download the PDF or CSV files directly.
- Each report honestly separates real measured numbers from any simulated/demo data used to fill gaps.

How we built it

1. Used a PDF-generation library to lay out charts and tables automatically from the day's data.
2. Reused the exact same waiting-time, fuel, and CO2 calculations from Section 5, so the reports always match what the live dashboard shows - no separate, inconsistent maths.

12. The Public Website

Project objectives: a modern public web portal for live traffic, signals, map, ambulance tracking, alerts, predictions and air quality

What it does

Gives any visitor - no login required - a clear, live view of the city's traffic, with six simple pages.

- **Live Overview** - totals, current modes, congestion per junction, air quality badges, and the latest public alerts, all in one glance.
- **Signals** - pick a junction and watch its animated traffic lights with a live countdown, plus vehicle counts and PCU per lane.
- **City Map** - an interactive map with junctions coloured by how congested they are, roads shown as open/congested/blocked, the ambulance's live position and route, any accidents, and detour roads highlighted in green.
- **Predictions** - the next-5/10-minute outlook per lane and the 24-hour peak-hour chart from Section 8.
- **Air Quality** - AQI per junction, pollutant breakdown, and a 24-hour trend.
- **Alerts** - a feed of public alerts: ambulance activity, accidents, congestion warnings, predictions, and air quality warnings.

What you'll actually see

- A dark-themed, modern-looking site (with a light mode toggle) that updates itself automatically every 2 seconds - open it and just watch the numbers move in real time.
- If the backend program isn't running, the site shows a clear 'Backend offline' banner instead of silently showing stale or broken data.

How we built it

1. Built using React (a popular tool for building interactive websites) and Tailwind CSS for consistent, modern styling.
2. Instead of needing a paid cloud database to 'push' live updates, the website simply asks the local backend 'what's new?' every 2 seconds - much simpler, completely free, and works fully offline on one laptop.
3. Used a free map library (OpenStreetMap via react-leaflet) so no paid map API key is needed.

13. The Controller's Control Room (Dashboard)

Project objectives: a centralised web controller dashboard to monitor all intersections, watch live video, and manage emergencies and accidents

What it does

A password-protected control room for traffic authority staff, with everything the public site has plus the tools to actually manage the system.

- **Command Centre** - key numbers at a glance, live video from the real junction, a grid of all junctions' mini signals, a live alert feed, and pending camera-spotted ambulance requests waiting for one-click approval.
- **Junction Detail** - live video, per-lane charts, predictions, current mode, and a **manual override** button to force a lane green by hand for a limited time (always safety-checked and reversible).
- **City Map** - the full map with an extra detail panel for controllers.
- **Emergencies** - start a trip, watch the active one live, and see trip history with time-saved statistics and an average-savings chart.
- **Accidents** - report or resolve an accident, see predicted impact, detours on a mini map, and evidence photos.
- **Violations** - a filterable, sortable table of every red-light violation with evidence photos and charts by lane and hour.
- **Predictions, Performance, Air Quality** - the same data as the public site, in more operational detail.
- **Reports** - generate and download the reports from Section 11.
- **Settings** - manage the list of authorised emergency vehicles, manage user accounts and roles, and see which AI model is currently deployed with its real accuracy numbers.

What you'll actually see

- Logging in as a controller unlocks a sidebar with all 11 sections above.
- Every write action (approving a request, overriding a signal, resolving an accident) is checked against the logged-in user's role before it's allowed - a driver account can never force a signal override, for example.

How we built it

1. Built the dashboard as extra pages in the same website, only visible after a successful login.
2. Every button that changes something (not just displays it) calls a specific, named backend action - never a generic 'write anything' call - so every permission can be checked individually and nothing slips through by accident.

14. Behind the Scenes - Login, Database & How It All Runs

Logging in safely

1. Passwords are never stored as plain text - they're scrambled using a standard, one-way scrambling method (PBKDF2) so that even looking directly at the database can't reveal anyone's real password.
2. After a correct login, the system hands back a signed 'token' (a JWT) - think of it as a stamped wristband proving who you are for the next 12 hours, checked on every single action without needing to log in again and again.

The database - one simple local file

1. All the data in the whole project - junctions, signals, live counts, violations, accidents, predictions, everything - lives in a single local file (SQLite), not a cloud service.
2. This project actually started on a free cloud database (Firebase), but that free tier has a daily limit on how many times you're allowed to save data - and heavy testing used it all up in a single day, freezing the live dashboard with no way to fix it until the next day. Switching to a local file removed that limit completely, for free, forever, with no internet dependency at all.
3. A single background 'writer' handles all saves one at a time, so nothing gets corrupted even with many parts of the program trying to save data at once.

How the whole thing runs

1. One single program (*main.py*) starts everything: the video-watching pipeline, the simulated extra junctions, the prediction scheduler, and the website's data server - all at once, in one command.
2. The website (*npm run dev*) is started separately and simply talks to that one program over the local network (localhost) - nothing ever leaves the computer unless you choose to put it online.
3. If any single piece fails (no camera feed, no GPU, a database hiccup), the rest of the system keeps running and shows a clear message instead of crashing everything else.

Simulated vs real, one more time for honesty: *the vehicle detection AI, the video processing, the PCU counting, the signal-timing logic, the violation detection, the prediction models, and the database are all real and genuinely running. The cameras (recorded video instead of live feeds), the traffic lights (drawn on screen instead of physical lights), the RFID reader (a typed tag ID instead of a physical scanner), GPS (a simulated moving ambulance), and the extra city junctions J2-J4 (simulated traffic counts) are deliberately simulated, because this is a software-only college project with no physical hardware budget - and that is stated clearly, not hidden.*

Nexiora

Smart Traffic Management System

A Beginner's Guide to Every Feature:
What It Does, How It Works, What You See, and How It Was Built

This guide explains the whole project in plain, simple language - no coding background needed. Every section follows the same pattern: what the feature does, how it works step by step, what you actually see on screen, and how it was built.

Honesty note: this is a college final-year project built without physical hardware. Cameras are recorded traffic videos, traffic lights are drawn on screen, RFID is a virtual tag check, GPS is a simulated ambulance, and the other city junctions are simulated. The vehicle detection, the AI model, and the counting are real and run on real video.

Contents

1. The Big Picture - How Everything Fits Together
2. Watching the Road - Vehicle Detection & Counting
3. Fair Counting - PCU-Weighted Traffic Load
4. Smart Traffic Signals - Adaptive Signal Control
5. Proving It Works - Fixed-Time Comparison & Savings
6. Catching Rule-Breakers - Red-Light Violation Detection
7. Ambulance Priority - RFID Check, Green Corridor & Nearest-Unit Alert
8. Seeing Into the Future - Traffic Prediction
9. Spotting Trouble - Accident Detection & Rerouting
10. Clean Air Matters - Air Quality & Eco Mode
11. Keeping Score - Reports & History
12. The Public Website
13. The Controller's Control Room (Dashboard)
14. Behind the Scenes - Login, Database & How It All Runs

1. The Big Picture

Think of Nexiora as a brain for a traffic junction. A camera (a recorded video, in this project) watches each of the 4 roads coming into the junction. The brain (a Python program) looks at the video, counts vehicles, decides which road should get a green light and for how long, watches for accidents and rule-breakers, and writes everything down in a notebook (a database). A website then reads that notebook and shows the results to two kinds of people:

- **The public** - anyone can open the website and see live traffic, the map, predictions, and air quality, with no login needed.
- **The controller** - a traffic authority staff member who logs in to see everything in detail, watch live video, approve ambulance requests, and override signals by hand if needed.

What you'll actually see

- Open the website and you immediately see 4 junction cards (J1-J4), each showing its traffic-light colours, how busy it is (Low/Medium/High), and the local air quality.
- Behind the scenes, this same information is refreshing automatically every 2 seconds - no page refresh needed.

How we built it, in simple steps:

1. Wrote a Python program that opens 4 traffic videos (one per road/lane) and reads them frame by frame, like flipping through a photo album very fast.
2. Fed each frame into an AI model that finds and counts vehicles (explained in Section 2).
3. Wrote a 'decision maker' that looks at the counts and decides signal timing (Section 4), checks for violations (Section 6) and accidents (Section 9).
4. Saved all the results into a simple local database file on the computer - no internet or cloud account needed.
5. Built a website (using React) that asks the database 'what's new?' every 2 seconds and redraws the screen with the latest numbers.

2. Watching the Road - Vehicle Detection & Counting

Project objectives: intelligent traffic monitoring, real-time vehicle counting per lane

What it does

Looks at each lane's video and figures out exactly how many vehicles are there right now, what type each one is (car, bike, auto-rickshaw, bus, or truck), and tracks each vehicle so it isn't counted twice as it moves across the screen.

How it works, step by step

1. A video frame (one photo from the video) is given to an AI model called **YOLOv8** - think of it as a very fast, trained 'spot the vehicle' game player. It draws a box around every vehicle it finds and labels what type it thinks it is.
2. This AI model was **trained specially for Indian traffic** using a real dataset called UVH-26 (thousands of labelled Indian road photos), so it recognises auto-rickshaws and two-wheelers, not just cars like a generic model would.
3. A second tool called **ByteTrack** gives each detected vehicle an ID number and follows it from frame to frame - like putting a sticky note with a number on every car the moment it appears, so the system knows 'this is the same car as last frame', not a new one.
4. A drawn boundary line (called an ROI - 'region of interest') marks the real lane area on screen. Only vehicles whose bottom-centre point is inside that boundary are counted for that lane - this stops vehicles in a neighbouring lane from being counted by mistake.

What you'll actually see

- On the live video feed, every vehicle has a green box drawn around it with its type and ID number, e.g. 'car #482'.
- Each lane shows a running total like '7 vehicles' directly on the video overlay and on the dashboard.
- If the camera feed is temporarily unavailable, the system says so clearly instead of freezing or crashing.

How we built it

1. Downloaded and converted the UVH-26 Indian traffic dataset into a format the AI model understands.
2. Trained a YOLOv8 model on a home GPU (laptop graphics card) to recognise 5 vehicle types: car, bike, auto, bus, truck.
3. Measured how accurate it is (a score called 'mAP' - about 90% accuracy overall) and compared it against an untrained, generic model to prove the training actually helped.
4. Connected the trained model to live video so it runs continuously, plus the ByteTrack tracker so each vehicle keeps one consistent ID as it moves.

3. Fair Counting - PCU-Weighted Traffic Load

What it does

Simply counting '5 vehicles' isn't fair - 5 bikes take up far less road space than 5 trucks. So the system also calculates a weighted number called **PCU** (Passenger Car Unit), a standard traffic-engineering idea: a car counts as 1 unit, a bike as 0.5 units (half a car's worth of road space), and a bus or truck as 3 units each.

How it works

1. Every detected vehicle already has a type label from Section 2 (car, bike, auto, bus, truck).
2. The system multiplies the count of each type by its PCU weight and adds them all up.
3. Example: 2 cars + 4 bikes + 1 truck = (2x1.0) + (4x0.5) + (1x3.0) = 7.0 PCU, even though the raw vehicle count is only 7 vehicles - here they happen to match, but a lane full of trucks will always show a much higher PCU than the same number of bikes.

What you'll actually see

- Every lane shows both numbers side by side, e.g. '6 vehicles, PCU 9.0' - so you can see both the raw count and the real road-space load.
- The junction cards show a congestion badge (Low / Medium / High) calculated from this PCU total, not just the raw count - a junction full of trucks is correctly shown as more congested than one full of bikes, even with the same vehicle count.

How we built it

1. Set the 5 PCU weight values in one central settings file, based on standard Indian Road Congress guidelines, so they can be tuned later without touching any other code.
2. Made every part of the system - the signal controller, the website, the predictions - read traffic 'load' from this same PCU number, so the whole project is consistent.

4. Smart Traffic Signals - Adaptive Signal Control

Project objectives: adaptive signal control - longer green for busy lanes, less waiting for quiet lanes

What it does

Instead of a traffic light that always gives each road exactly 30 seconds no matter what, this system looks at how busy each lane actually is right now and gives busier lanes a longer green light, and quiet lanes a shorter one - automatically, every cycle.

How it works, step by step

1. Every lane's current PCU load (Section 3) is fed into a formula: *green time = a base amount of seconds + extra seconds for every unit of PCU waiting*, with a minimum and maximum so no lane gets an unfairly short or endlessly long green.
2. After the green light's time runs out, a short yellow light warns drivers before the light turns red - this never changes, for safety.
3. The system then picks which lane goes green next: normally the lane with the most PCU waiting, but if any lane has been waiting too long (even a quiet one), it jumps the queue so no lane is ever ignored forever.
4. One safety rule is checked on every single update, with no exceptions: only one lane can ever be green or yellow at the same time. The other three are always red.
5. The system can also switch modes: normal ('adaptive'), rush hour ('peak', using the prediction feature), pollution-friendly ('eco'), emergency vehicle passing through, accident response, or a manual override by the human controller.

What you'll actually see

- On the video overlay and the dashboard, you see a traffic-light icon per lane with the real colour (red/yellow/green) and a countdown timer, e.g. 'Green: 17s left'.
- A plain-English log line is written for every decision the system makes, e.g. '[J1] lane3 -> GREEN for 22.0s (load=14.5, mode=adaptive)' - useful for explaining exactly why the system did what it did.

How we built it

1. Wrote a 'SignalController' program for each junction that holds the current state (which lane is green, for how long) and updates it once per second.
2. Wrote over 70 automated tests that check the safety rules can never be broken - for example, a test that tries to force two lanes green at once and checks the system refuses.
3. Connected the live PCU counts from Section 3 directly into this controller, so signal timing reacts to real traffic within seconds.

5. Proving It Works - Fixed-Time Comparison & Savings

Project objectives: measurably reduce waiting time, idle fuel and emissions

What it does

It's not enough to say 'our smart signals are better' - you have to prove it with numbers. So a second, imaginary traffic light runs silently alongside the real one, using the old-fashioned fixed-time method (always the same number of seconds per lane) on the exact same traffic. The system then compares the two.

How it works

1. Both the adaptive (smart) and fixed-time (old-fashioned) signal timers watch the same live vehicle counts, but make their own independent decisions - the fixed-time one never actually controls the real lights, it only runs the maths in the background for comparison.
2. Every second, every vehicle sitting at a red or yellow light adds one 'waiting vehicle-second' to that method's running total.
3. Average waiting time = total waiting vehicle-seconds divided by how many vehicles were served - a simple, honest average.
4. Wasted fuel and CO2 are estimated from the waiting time, using a standard idling-fuel-burn rate (a labelled assumption, not a measured fact, since there's no real fuel sensor).

What you'll actually see

- The dashboard's Performance page shows a side-by-side chart: 'Adaptive: 12.4s average wait' vs 'Fixed-time: 19.8s average wait', plus estimated litres of fuel and kilograms of CO2 saved per day.
- These are real, measured numbers from the actual running system, not invented - the project is careful to only claim savings it can show the maths for.

How we built it

1. Built a simple fixed-time controller that always gives every lane the same cycle, for a fair 'before' comparison.
2. Built a shared waiting-time tracker used by both controllers, so the comparison is apples-to-apples.
3. Set the fuel/CO2 conversion numbers in the settings file and labelled them clearly as assumptions in the project's documentation, as required by academic honesty.

6. Catching Rule-Breakers - Red-Light Violation Detection

Project objectives: AI red-light violation detection with recorded evidence

What it does

Automatically catches any vehicle - car, bike, auto, bus or truck - that crosses the stop line while its lane's light is red, takes a photo as evidence, and logs it with a unique vehicle tracking number.

How it works, step by step

1. A stop-line position is marked for each lane (one pixel row on the video).
2. For every tracked vehicle, the system remembers where its bottom edge was in the previous frame and where it is now.
3. If that bottom edge was above the stop line last frame and is now past it **while the light is red**, that's a violation - caught at the exact moment of crossing, not before or after.
4. An authorised emergency vehicle that's currently been given priority is never flagged, even if it must cross on red to get through.
5. The same vehicle is only ever flagged once, even if it lingers near the line.

What you'll actually see

- On the video, the violating vehicle gets a red box and the word 'VIOLATION' written above it for a few seconds.
- A full snapshot of the moment, plus a close-up crop of just the vehicle, is saved as image files - real evidence, not just a log line.
- A new row is added to the database and shown on the controller dashboard's Violations page: lane, vehicle type (car/bike/auto/bus/truck), a random/unique tracking number, and timestamp - filterable and exportable as a table with charts by lane and time of day.

How we built it, and a real fix we made

1. Wrote a 'RedLightDetector' that watches every tracked vehicle's position each frame and compares it to the stop line and the current light colour.
2. Added an evidence-saving step using the computer's image library to crop and save the photo the instant a violation is confirmed.
3. Added a 'demo mode' that holds one lane red for 30 seconds on purpose, so a violation can be shown on stage during a presentation without waiting for a real red light.

7. Ambulance Priority - RFID Check, Green Corridor & Nearest-Unit Alert

Project objectives: automatic priority for authorised emergency vehicles, live ambulance tracking with a green corridor, faster emergency response

What it does

Lets a real, registered ambulance get a clear run of green lights across the city - but **only** after proving it's a genuine, authorised vehicle, not just because something that looks like an ambulance appears on camera. This is split into three connected parts.

Part A - Checking the ambulance is real (virtual RFID)

1. Since there's no physical RFID scanner in this project, a driver 'logs in' as a specific ambulance and enters a tag ID number - this stands in for scanning a real RFID chip on the vehicle.
2. An emergency request can also come from the camera itself spotting something that looks like an ambulance, from the controller's dashboard, or from a simulator script - but every single one of these is only a **request**, nothing more.
3. The system checks the vehicle ID and tag ID against a list of registered emergency vehicles. Only an exact match gets approved; everything else is rejected with a reason shown to the driver, and a security alert is raised.
4. This matters: a camera alone can never grant priority across multiple junctions - only a verified tag can. This stops anyone from just pretending to be an ambulance to skip traffic.

Part B - Clearing the road ahead (green corridor)

1. Once approved, the system calculates the fastest route to the destination, avoiding any roads currently blocked by an accident.
2. For each junction coming up on that route, it estimates: how many seconds until the ambulance arrives, and how many seconds that junction's own queue needs to clear out of the way.
3. If the ambulance will arrive before the queue would naturally clear, that junction switches early to give the ambulance's direction a green light - clearing the jam before it gets there, not panicking at the last second.
4. For safety, the light always passes through yellow first, never straight from red to green, and at most the next 2 junctions ahead are held at once.
5. Once the ambulance has passed, that junction goes back to normal and serves whichever lane has been waiting longest.

Part C - Telling the nearest ambulance about an accident

1. When an accident is confirmed (Section 9), the system looks at every registered emergency vehicle's home base and calculates the straight-line distance from each one to the accident.
2. The closest available unit gets a private notification on their driver screen: 'Accident near [junction name] (1.2 km from your base) - you're the nearest available unit.'

3 This closes the loop - the system doesn't just detect an accident, it tells the right people to go deal with it.

What you'll actually see

- The driver's screen shows: pending -> approved (with route, live map, and the list of junctions being held green for them) -> or rejected (with a clear reason, e.g. unknown vehicle ID).
- The public map shows the ambulance's live position moving along its route, with an on-screen banner at any junction currently being held for it.
- After the trip, a record is saved showing how many seconds were saved compared to no green corridor at all - this can be averaged over many simulated trips into a chart for the project report.

How we built it

1 Seeded a small list of registered ambulances/fire vehicles with tag IDs and a home junction.

2 Wrote a route-finder using the city's road map (modelled as a graph of junctions and roads) that always finds the fastest currently-open path.

3 Wrote a 'GreenCorridor' program that ticks once a second, recalculating ETA vs queue clearance time for every junction on the route and deciding when to pre-empt.

4 Fixed a real bug found while testing: the green corridor could only ever reach the main junction's traffic light, not the simulated ones elsewhere in the city, because they lived in a separate part of the program - connected them properly so priority now works across the whole simulated city, verified with a real end-to-end test run.

8. Seeing Into the Future - Traffic Prediction

Project objectives: ML prediction of future congestion - peak hours and next 5-10 minutes

What it does

Uses machine learning (a computer program that learns patterns from past data, rather than being told exact rules) to guess two different things: which hours of the day are usually the busiest, and what traffic will look like in the next 5 to 10 minutes.

How it works - Part A: Peak hours

1. The system keeps a history of traffic counts logged every minute, plus 30 days of realistic sample data generated to fill in gaps (clearly labelled as simulated, not pretending to be real).
2. Two different learning methods are tried - a simple one (Linear Regression) and a more flexible one (Random Forest) - and their accuracy is measured and compared honestly.
3. The better method predicts traffic for all 24 hours of the next day; the busiest 20% of those hours are marked as 'peak hours'.
4. When a peak hour arrives, the signal controller automatically switches to 'peak' mode (Section 4) to get ahead of the rush instead of reacting to it.

How it works - Part B: Next 5-10 minutes

1. The system looks at each lane's last several minutes of traffic (how it's trending: rising, falling, steady) plus the time of day and day of week.
2. It predicts whether that lane will be Low, Medium, or High in 5 minutes and in 10 minutes, refreshing this prediction every 60 seconds.
3. If a lane is predicted to go 'High' soon, the signal controller adds a few bonus seconds to that lane's next green light *before* it actually gets busy - preventing the jam rather than just reacting to it.

What you'll actually see

- The Predictions page shows a 24-hour chart with the predicted busy hours highlighted.
- Each lane shows a simple tag like 'In 5 min: Medium, In 10 min: High'.
- An alert pops up (at most once every 10 minutes per lane, so it doesn't spam) warning that a lane is about to get busy.

How we built it

1. Generated realistic synthetic traffic history (with believable rush-hour patterns) since there wasn't months of real historical data available.
2. Trained and compared Linear Regression and Random Forest models for both the hourly and minute-level predictions, measuring their error honestly (MAE, RMSE, R-squared - standard accuracy scores) instead of just picking one and assuming it works.
3. Connected the winning model's live predictions back into the signal controller, so prediction isn't just a chart - it actually changes real signal decisions.

9. Spotting Trouble - Accident Detection & Rerouting

Project objectives: accident detection, congestion-impact prediction, alternative-route suggestions

What it does

Watches the video for signs that look like an accident, warns the controller, predicts how much extra traffic delay it will cause, and automatically suggests detours around the blocked road.

How it works, step by step

1. **Rule 1 - stuck vehicle:** if a vehicle stays stationary while its own light is green for an unusually long time, that's suspicious - a vehicle that's fine would have driven off on green.
2. **Honest limitation fixed:** the recorded video has its own real-world traffic lights that this project's virtual signal doesn't know about, so a car legitimately stopped at a real red light used to look identical to a stuck car. The fix: a suspected vehicle now has to survive a *full green-red-green cycle* of the virtual signal still not moving before it's confirmed as a real accident - a real red-light stop clears well before that, but a genuinely stuck vehicle doesn't.
3. **Rule 2 - sudden collision:** if two tracked vehicles suddenly overlap heavily and both sharply drop speed within about a second, that pattern also raises suspicion.
4. A suspicion only becomes a confirmed accident after it holds up for a set number of seconds, to avoid false alarms from a single bad camera frame.
5. Once confirmed: the road is marked blocked, that approach switches to 'accident' mode, alternative routes are calculated for traffic that would have used that road, an alert is raised with the suggested detour, and any active ambulance route is automatically re-planned around the blockage.
6. The controller can also manually report or resolve an accident with one click, and a simulator can trigger a practice accident for demo purposes.

What you'll actually see

- The video shows an orange 'ACCIDENT?' box the moment something looks suspicious, and a full red accident banner once it's confirmed.
- A snapshot image is saved as evidence.
- The dashboard's Accidents page shows active and resolved accidents, the predicted extra delay in minutes, the suggested detour on a mini map, and a 'Mark resolved' button.
- The nearest ambulance also gets notified automatically (Section 7, Part C).

How we built it

1. Wrote simple, explainable rules rather than training a separate AI model, since no accident-image dataset was available - and said so honestly in the project documentation rather than pretending otherwise.
2. Added the 'must survive a full signal cycle' rule after watching the system raise false accident alarms during real testing, then wrote an automated test that specifically proves a car stuck only during one red-light stop is never falsely confirmed.

3 Connected confirmed accidents to the road-network router, so detours are calculated the same way as any other route in the system, not as a separate one-off feature.

10. Clean Air Matters - Air Quality & Eco Mode

Project objectives: air-quality monitoring and eco-friendly control

What it does

Checks the air quality near each junction and, when pollution is high, switches the signal controller into a gentler 'eco' mode that reduces how long vehicles sit idling with their engines running.

How it works

1. Every few minutes, the system fetches real air quality data (PM2.5, PM10, NO2, CO) from a free weather-and-pollution service for each junction's location; if that service is unavailable, it clearly falls back to a labelled simulated reading instead of pretending to have real data.
2. The reading is converted into a simple AQI (Air Quality Index) score and a plain label like 'Fair' or 'Moderate'.
3. If the AQI reaches a poor level, the junction automatically switches to 'eco' mode: shorter overall cycles and a lower maximum-wait limit, so fewer vehicles sit idling and polluting for a long time.
4. An 'air quality' alert is raised, but at most once per hour per junction, so it doesn't spam the alert feed.

What you'll actually see

- Every junction card shows an AQI badge, e.g. 'AQI 2 (Fair)'.
- The Air Quality page shows pollutant levels and a 24-hour trend chart per junction.
- When eco mode kicks in, the junction's mode badge changes to 'eco' and the dashboard shows this clearly.

How we built it

1. Signed up for a free tier of a weather/pollution API and stored the key securely outside the code (never hard-coded or shared).
2. Wrote a fallback simulator so the feature still demonstrates correctly even without an internet connection or API key.
3. Added an 'EcoModeManager' that checks AQI on a timer and flips the right junction's signal controller into eco mode automatically.

11. Keeping Score - Reports & History

Project objectives: historical records and reports on traffic patterns and system performance

What it does

Turns all the raw data the system has collected into readable daily summaries, charts, and downloadable reports - useful for a project viva, or for a real traffic authority reviewing performance over time.

How it works

1. A report-building script gathers everything recorded for a chosen day (or date range): traffic counts, wait times, violations, accidents, emergency trips, and air quality.
2. It produces a professional PDF with charts and captions, plus raw CSV spreadsheet files for anyone who wants to analyse the numbers themselves.
3. A 'fill in demo data' option can generate a realistic past week of records in one go, useful for showing the reports feature without needing to run the live system for a whole week first.

What you'll actually see

- The controller dashboard's Reports page lists daily summaries with a trend chart, and buttons to download the PDF or CSV files directly.
- Each report honestly separates real measured numbers from any simulated/demo data used to fill gaps.

How we built it

1. Used a PDF-generation library to lay out charts and tables automatically from the day's data.
2. Reused the exact same waiting-time, fuel, and CO2 calculations from Section 5, so the reports always match what the live dashboard shows - no separate, inconsistent maths.

12. The Public Website

Project objectives: a modern public web portal for live traffic, signals, map, ambulance tracking, alerts, predictions and air quality

What it does

Gives any visitor - no login required - a clear, live view of the city's traffic, with six simple pages.

- **Live Overview** - totals, current modes, congestion per junction, air quality badges, and the latest public alerts, all in one glance.
- **Signals** - pick a junction and watch its animated traffic lights with a live countdown, plus vehicle counts and PCU per lane.
- **City Map** - an interactive map with junctions coloured by how congested they are, roads shown as open/congested/blocked, the ambulance's live position and route, any accidents, and detour roads highlighted in green.
- **Predictions** - the next-5/10-minute outlook per lane and the 24-hour peak-hour chart from Section 8.
- **Air Quality** - AQI per junction, pollutant breakdown, and a 24-hour trend.
- **Alerts** - a feed of public alerts: ambulance activity, accidents, congestion warnings, predictions, and air quality warnings.

What you'll actually see

- A dark-themed, modern-looking site (with a light mode toggle) that updates itself automatically every 2 seconds - open it and just watch the numbers move in real time.
- If the backend program isn't running, the site shows a clear 'Backend offline' banner instead of silently showing stale or broken data.

How we built it

1. Built using React (a popular tool for building interactive websites) and Tailwind CSS for consistent, modern styling.
2. Instead of needing a paid cloud database to 'push' live updates, the website simply asks the local backend 'what's new?' every 2 seconds - much simpler, completely free, and works fully offline on one laptop.
3. Used a free map library (OpenStreetMap via react-leaflet) so no paid map API key is needed.

13. The Controller's Control Room (Dashboard)

Project objectives: a centralised web controller dashboard to monitor all intersections, watch live video, and manage emergencies and accidents

What it does

A password-protected control room for traffic authority staff, with everything the public site has plus the tools to actually manage the system.

- **Command Centre** - key numbers at a glance, live video from the real junction, a grid of all junctions' mini signals, a live alert feed, and pending camera-spotted ambulance requests waiting for one-click approval.
- **Junction Detail** - live video, per-lane charts, predictions, current mode, and a **manual override** button to force a lane green by hand for a limited time (always safety-checked and reversible).
- **City Map** - the full map with an extra detail panel for controllers.
- **Emergencies** - start a trip, watch the active one live, and see trip history with time-saved statistics and an average-savings chart.
- **Accidents** - report or resolve an accident, see predicted impact, detours on a mini map, and evidence photos.
- **Violations** - a filterable, sortable table of every red-light violation with evidence photos and charts by lane and hour.
- **Predictions, Performance, Air Quality** - the same data as the public site, in more operational detail.
- **Reports** - generate and download the reports from Section 11.
- **Settings** - manage the list of authorised emergency vehicles, manage user accounts and roles, and see which AI model is currently deployed with its real accuracy numbers.

What you'll actually see

- Logging in as a controller unlocks a sidebar with all 11 sections above.
- Every write action (approving a request, overriding a signal, resolving an accident) is checked against the logged-in user's role before it's allowed - a driver account can never force a signal override, for example.

How we built it

1. Built the dashboard as extra pages in the same website, only visible after a successful login.
2. Every button that changes something (not just displays it) calls a specific, named backend action - never a generic 'write anything' call - so every permission can be checked individually and nothing slips through by accident.

14. Behind the Scenes - Login, Database & How It All Runs

Logging in safely

1. Passwords are never stored as plain text - they're scrambled using a standard, one-way scrambling method (PBKDF2) so that even looking directly at the database can't reveal anyone's real password.
2. After a correct login, the system hands back a signed 'token' (a JWT) - think of it as a stamped wristband proving who you are for the next 12 hours, checked on every single action without needing to log in again and again.

The database - one simple local file

1. All the data in the whole project - junctions, signals, live counts, violations, accidents, predictions, everything - lives in a single local file (SQLite), not a cloud service.
2. This project actually started on a free cloud database (Firebase), but that free tier has a daily limit on how many times you're allowed to save data - and heavy testing used it all up in a single day, freezing the live dashboard with no way to fix it until the next day. Switching to a local file removed that limit completely, for free, forever, with no internet dependency at all.
3. A single background 'writer' handles all saves one at a time, so nothing gets corrupted even with many parts of the program trying to save data at once.

How the whole thing runs

1. One single program (*main.py*) starts everything: the video-watching pipeline, the simulated extra junctions, the prediction scheduler, and the website's data server - all at once, in one command.
2. The website (*npm run dev*) is started separately and simply talks to that one program over the local network (localhost) - nothing ever leaves the computer unless you choose to put it online.
3. If any single piece fails (no camera feed, no GPU, a database hiccup), the rest of the system keeps running and shows a clear message instead of crashing everything else.

Simulated vs real, one more time for honesty: *the vehicle detection AI, the video processing, the PCU counting, the signal-timing logic, the violation detection, the prediction models, and the database are all real and genuinely running. The cameras (recorded video instead of live feeds), the traffic lights (drawn on screen instead of physical lights), the RFID reader (a typed tag ID instead of a physical scanner), GPS (a simulated moving ambulance), and the extra city junctions J2-J4 (simulated traffic counts) are deliberately simulated, because this is a software-only college project with no physical hardware budget - and that is stated clearly, not hidden.*